SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA05**

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 20%

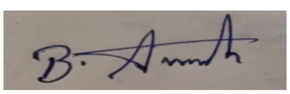
Read the given scenario carefully and analyze the problem using appropriate computer vision techniques. Justify your answers with relevant reasoning and comparisons wherever necessary.

<u>Code of Conduct:</u>

I Avinash B (192511193) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Note: As requested, this submission answers questions 1, 2, 3, 4, and 6 only.

Question 1: Preprocessing and Feature Detection

Scenario: A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent.

Analysis of the issue

Most feature detectors (Harris corners, SIFT, edge detectors, etc.) locate features by measuring local intensity gradients or contrast. Under varying illumination — shadows, overexposure, underexposure, or uneven lighting across the scene — the raw pixel intensities of the same physical feature shift non-uniformly between images. A corner or edge that produces a strong, clearly-above-threshold gradient response in a well-lit image may produce a weak, inconsistent, or below-threshold response in a poorly lit version of the same scene. As a result, the detector finds a different set of features (or localizes them differently) each time, even though the underlying scene content has not actually changed — this is what causes the inconsistent detection results described.

How preprocessing improves feature detection performance

- Histogram equalization / adaptive histogram equalization (CLAHE): redistributes intensity values so contrast is standardized across the image, including in individually dark or bright regions, compensating for uneven exposure.
- Gaussian smoothing: suppresses small-scale sensor/lighting noise that would otherwise generate spurious gradient responses unrelated to real scene structure.
- Illumination normalization (gamma correction, homomorphic filtering, retinex-based methods): separates the illumination component from the underlying reflectance of the scene, correcting global or local brightness variation before features are extracted.
- Gradient-based normalization: since most detectors and descriptors respond to relative intensity change rather than absolute intensity, normalizing gradients further reduces sensitivity to the overall lighting level.

Together, these steps ensure the same physical feature produces a similar detector response regardless of the lighting under which it was captured, which directly improves repeatability (the same features being found across different images of the same scene) and the precision of feature localization.

Question 2: Edge vs Corner Detection

Scenario: In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching.

Why edge detection is insufficient

Edge detectors (e.g., Sobel, Canny) mark pixels where intensity changes sharply along one direction — they trace boundaries/contours, but do not, by themselves, produce distinctive points. Critically, every point along a straight edge looks locally identical to every other point along that same edge (this is the classical 'aperture problem'): sliding a small window along the edge direction produces no change in local appearance. Because a matching algorithm needs a point whose local neighborhood is uniquely identifiable, a generic point on a straight edge cannot be reliably or uniquely matched between two images — there is nothing to distinguish one position along the edge from its neighbors.

Justification for using corner detection

A corner is a location where the intensity changes sharply in two (or more) directions at once. Detectors such as the Harris corner detector formalize this using the local structure tensor (autocorrelation matrix): a true corner is identified where both eigenvalues of this matrix are large, meaning a small shift in any direction produces a significant change in local appearance. This makes corners well-localized in both image dimensions, distinctive, and repeatable — exactly the properties a matching algorithm needs to uniquely identify and re-locate the same point across two images. In a scene with straight edges and textured regions, corners naturally arise at edge intersections and fine texture detail, providing exactly the kind of unambiguous keypoints that plain edge pixels cannot offer, which is why corner detection (rather than edge detection alone) is the appropriate technique here.

Question 3: Hough Transform in Noisy Images

Scenario: A system uses the Hough Transform to detect lines in noisy images but produces inaccurate results.

Effect of noise on detection

The Hough Transform works by having every edge pixel 'vote' in a parameter space (ρ, θ for lines), with genuine lines appearing as strong peaks where many collinear edge points' votes accumulate. Noise disrupts this process in several ways:

- Spurious edge pixels introduced by noise cast votes scattered throughout the parameter space; if enough noisy points happen to be roughly collinear by coincidence, they can create false peaks — detected 'lines' that do not correspond to any real line in the image.
- Noise also corrupts and fragments the edge map along genuine lines (some true edge pixels are lost or shifted), so fewer votes accumulate at the true line's peak, weakening it relative to noise-induced peaks and increasing the risk of missed detections.
- Small positional errors in noisy edge pixels spread votes across several adjacent (ρ, θ) bins near the true line, which can split what should be one strong peak into several smaller, less confident peaks — causing duplicate or slightly misaligned line detections instead of one accurate one.

Suggested improvements

Preprocessing: apply Gaussian smoothing or a median filter (particularly effective against salt-and-pepper noise) before edge detection to suppress spurious gradient responses; use Canny edge detection with properly tuned hysteresis thresholds so weak, noise-driven edges are discarded while genuine strong edges are retained and connected; apply morphological closing to reconnect edge fragments broken by noise.

Parameter selection: tune the Hough accumulator's voting threshold so that noise-driven low-vote peaks are filtered out without discarding genuine (but slightly weaker) lines; choose an appropriate (ρ, θ) bin resolution — too fine scatters votes from noisy points across many cells and weakens true peaks, too coarse loses angular precision; where possible, use a gradient-direction-weighted or probabilistic Hough Transform, which restricts each edge pixel's votes to angles consistent with its local gradient direction (rather than voting across all possible θ), sharply reducing the spread caused by noise and producing cleaner, more accurate peaks.

Question 4: Feature Matching under Transformations

Scenario: Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale-invariant feature matching works effectively.

Why scale-invariant methods perform better

Traditional matching approaches compare local image patches or descriptors computed over a fixed-size window at a fixed orientation. When the object appears at a different size (scale change) or is rotated between the two images, the raw pixel neighborhood around a truly corresponding point looks substantially different — the same physical feature now spans a different number of pixels, and local gradient directions are rotated relative to the fixed reference frame the descriptor was computed in. Comparing these mismatched, fixed-window descriptors produces low similarity scores even for genuinely corresponding points, so matching fails or returns many incorrect correspondences.

Scale-invariant methods such as SIFT address this directly:

- Scale-space construction: keypoints are detected across a pyramid of progressively blurred/resized images (e.g., a Difference-of-Gaussian pyramid), and each keypoint is assigned its own characteristic scale — the scale at which it produces a local extremum in this pyramid. The same physical feature is therefore detected at a scale that automatically adapts to how large or small the object appears in each image.
- Orientation assignment: each keypoint is given a dominant orientation based on the local gradient directions around it, and its descriptor is computed relative to that orientation — so rotating the whole image simply rotates the reference frame the descriptor uses, leaving the descriptor values themselves unchanged.
- Scale-normalized sampling: the descriptor's sampling region is sized according to the keypoint's detected scale, so the descriptor always captures the same relative image content regardless of the image's zoom level.

Because each keypoint's descriptor is expressed relative to its own detected scale and orientation rather than a fixed global window, corresponding physical features produce nearly identical descriptor vectors across the two images even though the raw images differ in scale and rotation — enabling reliable nearest-neighbor matching where fixed-window traditional methods fail.

Question 6: Preprocessing Impact on Edge Detection

Scenario: A system performs edge detection on raw images containing noise and uneven illumination, resulting in fragmented edges.

Analysis of the problem

Edge detectors identify boundaries by locating locally strong, consistent intensity gradients. Two compounding problems cause fragmentation here:

- Noise superimposes random small-scale intensity fluctuations across the image. Along a true edge, this causes the local gradient magnitude to fluctuate above and below the detector's threshold at different points — some parts of a continuous real boundary are marked as edges while other parts (where noise happens to locally reduce the gradient) fall below threshold and are missed, breaking one continuous boundary into disconnected segments.
- Uneven illumination compounds this systematically rather than randomly: the same physical boundary produces a smaller intensity difference (lower inherent contrast) in poorly lit regions of the image than in well-lit regions. This pushes edge segments in darker areas below threshold even when the noise level is otherwise unchanged, producing gaps that correlate with the lighting pattern across the image rather than occurring uniformly.

How preprocessing improves accuracy and continuity

- Denoising (Gaussian blur, median filtering, or edge-preserving filters such as bilateral filtering) suppresses random noise-driven gradient fluctuations before edge detection, keeping the true edge's gradient magnitude more consistently above threshold along its full length.
- Illumination correction (histogram equalization/CLAHE, homomorphic filtering, or estimating and subtracting a smooth illumination field) normalizes contrast across the image, so a genuine boundary produces a comparable gradient response whether it lies in a bright or dark region — removing the systematic, lighting-correlated fragmentation.
- Using hysteresis-based edge detection (Canny) after this preprocessing, with properly tuned dual thresholds, allows weaker-but-genuine edge segments to be retained as long as they connect to a strong edge segment, further improving continuity.
- Morphological post-processing (dilation followed by thinning, or explicit edge-linking) can bridge any small remaining gaps once preprocessing has already removed most of the noise- and illumination-driven fragmentation.

Together, these steps give the edge detector a cleaner, more spatially uniform gradient signal to threshold against, producing longer, more continuous, and more accurate edge contours that better represent the scene's true object boundaries.